Experiment No. 4
Train a Variational Autoencoder on Fashion MNIST.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Train a Variational Autoencoder on Fashion MNIST.

Objective:

• To understand the concept of Autoencoders and Variational Autoencoders (VAE).
• To implement VAE for image generation and representation learning.
• To train the model on Fashion MNIST dataset.
• To analyze latent space representation of images.

Theory:

Autoencoder:
An Autoencoder is a neural network used for learning efficient data representations. It consists of:
Encoder: Compresses input into latent representation
Decoder: Reconstructs input from latent representation

Variational Autoencoder (VAE):
VAE is a probabilistic version of an autoencoder that learns a distribution instead of fixed encoding.

Instead of mapping input to a single point, it maps to:

Mean (μ)

Variance (σ^2)

A latent variable is sampled from this distribution.

Key Concepts:
Latent Space: Compressed representation of data
Reparameterization Trick: Allows backpropagation through sampling
Loss Function:
Reconstruction Loss

KL Divergence Loss

Working:

1. Input image is encoded into mean and variance
2. Latent vector is sampled
3. Decoder reconstructs the image
4. Model learns to generate similar images

Implementation:

• Import required libraries such as TensorFlow/Keras and NumPy.

• Load and preprocess Fashion MNIST dataset (normalize images).

• Define Encoder network to generate mean and variance.

• Apply reparameterization trick to sample latent vector.

• Define Decoder network to reconstruct images.

• Define loss function:

Reconstruction loss

KL divergence

• Train the VAE model on dataset.

• Generate new images from latent space.

Code:

 # Install dependencies:

 # pip install tensorflow numpy matplotlib

import tensorflow as tf

from tensorflow.keras import layers, Model

```
import numpy as np

import matplotlib.pyplot as plt


# Step 1: Load and preprocess Fashion MNIST

(x_train, _), (x_test, _) = tf.keras.datasets.fashion_mnist.load_data()


x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


x_train = np.reshape(x_train, (-1, 28, 28, 1))

x_test = np.reshape(x_test, (-1, 28, 28, 1))


latent_dim = 2


# Step 2: Encoder

inputs = layers.Input(shape=(28, 28, 1))

x = layers.Conv2D(32, 3, activation="relu", strides=2, padding="same")(inputs)

x = layers.Conv2D(64, 3, activation="relu", strides=2, padding="same")(x)

x = layers.Flatten()(x)

x = layers.Dense(16, activation="relu")(x)


z_mean = layers.Dense(latent_dim, name="z_mean")(x)
```

```
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)


# Step 3: Reparameterization Trick

def sampling(args):

    z_mean, z_log_var = args

    epsilon = tf.random.normal(shape=(tf.shape(z_mean)[0], latent_dim))

    return z_mean + tf.exp(0.5 * z_log_var) * epsilon


z = layers.Lambda(sampling, output_shape=(latent_dim,))([z_mean, z_log_var])


encoder = Model(inputs, [z_mean, z_log_var, z], name="encoder")


# Step 4: Decoder

latent_inputs = layers.Input(shape=(latent_dim,))

x = layers.Dense(7*7*64, activation="relu")(latent_inputs)

x = layers.Reshape((7, 7, 64))(x)


x = layers.Conv2DTranspose(64, 3, strides=2, padding="same", activation="relu")(x)

x = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(x)

outputs = layers.Conv2DTranspose(1, 3, activation="sigmoid", padding="same")(x)


decoder = Model(latent_inputs, outputs, name="decoder")
```

```
# Step 5: VAE Model
class VAE(Model):
    def __init__(self, encoder, decoder):
        super(VAE, self).__init__()
        self.encoder = encoder
        self.decoder = decoder

    def train_step(self, data):
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)

            # Reconstruction Loss
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(
                    tf.keras.losses.binary_crossentropy(data, reconstruction), axis=(1,2)
                )
            )

            # KL Divergence Loss
            kl_loss = -0.5 * tf.reduce_mean(
```

```
        tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=1)
    )

    total_loss = reconstruction_loss + kl_loss

    grads = tape.gradient(total_loss, self.trainable_weights)
    self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

    return {
        "loss": total_loss,
        "reconstruction_loss": reconstruction_loss,
        "kl_loss": kl_loss,
    }

vae = VAE(encoder, decoder)
vae.compile(optimizer=tf.keras.optimizers.Adam())

# Step 6: Train Model
vae.fit(x_train, epochs=20, batch_size=128, validation_data=(x_test, None))

# Step 7: Generate Images
def generate_images():
```

```
    z_sample = np.random.normal(size=(16, latent_dim))

    generated = decoder.predict(z_sample)



    plt.figure(figsize=(4,4))

    for i in range(16):

        plt.subplot(4,4,i+1)

        plt.imshow(generated[i].reshape(28,28), cmap='gray')

        plt.axis('off')

    plt.show()


generate_images()
```

Conclusion:

Variational Autoencoders are powerful generative models that learn data distributions and generate new samples. They are useful for image generation, feature learning, and dimensionality reduction.